

```

del aunay.remove(invalid_triangles[i])
for j in range(0,len(invalid_triangles[i])):
    points_in_invalid.append(invalid_triangles[i][j])

```

❶ `points_in_invalid = [list(x) for x in set(tuple(x) for x in points_in_invalid)]`

We first create an empty list called `points_in_invalid`. Then, we loop through `invalid_triangles`, using Python's `remove()` method to take each invalid triangle out of the existing DT. We then loop through every point in the triangle to add it to the `points_in_invalid` list. Finally, since we may have added some duplicate points to the `points_in_invalid` list, we'll use a list comprehension ❶ to re-create `points_in_invalid` with only unique values.

The final step in our algorithm is the trickiest one. We have to add new triangles to replace the invalid ones. Each new triangle will have the `point_to_add` as one of its points, and and two points from the existing DT as its other points. However, we can't add every possible combination of `point_to_add` and two existing points.

In Figures [7-13](#) and [7-14](#), notice that the new triangles we needed to add were all triangles with point 10 as one of their points, and with edges selected from the empty polygon that contained point 10. This may seem simple enough after a visual check, but it's not straightforward to write code for it.

We need to find a simple geometric rule that can be easily explained in Python's hyper-literal style of interpretation. Think of the rules that could be used to generate the new triangles in [Figure 7-14](#). As is common in mathematical situations, we could find multiple equivalent sets of rules. We could have rules related to points, since one definition of a triangle is a set of three points. We could have other rules related to lines, since another, equivalent definition of triangles is a set of three line segments. We could use any set of rules; we just want the one that will be the simplest to understand and implement in our code. One possible rule is that we should consider every possible combination of points in the invalid triangles with the `point_to_add`, but we should add one of those triangles only if the edge not containing the `point_to_add` occurs exactly once in the list of invalid triangles. This rule works because the edges that occur exactly once will be the edges of the outer polygon surrounding the new point (in [Figure 7-13](#), the edges in question are the edges of the polygon connecting points 1, 4, 8, 7, and 3).